EFREI Paris

ST2OOS: Object-Oriented Systems Development

Academic year 2026/2027

QuantBacktest

Financial Strategy Backtesting Platform

Project report

Alexandre Tung	Data, reports, deployment
Victor Haegeman	Strategies, no-code rules, Monte Carlo
Eloi Dumas	Backtesting engine, metrics, optimisation
Alexis Launay-Schapman	Web application

Source code:
`https://github.com/Alexarmanto/QuantBacktest-Financial-Strategy-Backtesting-Platform`

Submitted for 14 November 2026

Contents

1 Introduction **4**

1.1 Objectives 4

1.2 Scope and summary of features 4

1.3 Organisation of the work 5

2 Architecture **6**

2.1 Layers 6

2.2 The contract 6

2.3 The golden rule: no project knows another 6

2.4 Request flow 7

2.5 Fakes and parallel work 7

2.6 Conventions that avoided bugs 7

3 Data layer **9**

3.1 What the free plan really gives 9

3.2 Cache, seed files and the loading algorithm 9

3.3 Talking to Alpha Vantage safely 10

3.4 Parsing, quota and concurrency 11

3.5 Data quality and API quota 11

4 Strategies **13**

4.1 What a strategy is allowed to do 13

4.2 Indicators 13

4.3 The built-in strategies 13

4.4 A first look at the results 15

4.5 Writing a strategy in C# 16

4.6 Building a strategy without code 16

5 Backtesting engine **18**

5.1 Execution model . . . 18

5.2 Advanced orders . . . 18

5.3 Metrics . . . 19

5.4 Validation of requests and errors . . . 19

5.5 Optimisation heatmap . . . 20

5.6 How we checked the numbers . . . 21

6 Robustness analysis **23**

6.1 Train/test validation . . . 23

6.2 Monte Carlo simulation . . . 24

7 Web application **26**

7.1 Pages and user journey . . . 26

7.2 Design choices . . . 27

7.3 Comparator . . . 28

7.4 Permanent links . . . 28

8 Reports and deployment **30**

8.1 PDF report . . . 30

8.2 Docker . . . 32

9 Quality and teamwork **33**

9.1 Tests . . . 33

9.2 Git workflow and continuous integration . . . 33

9.3 Use of AI assistants . . . 34

9.4 What we learned . . . 35

10 Limitations and future work **36**

10.1 Data, reports and deployment . . . 36

10.2 Strategies and Monte Carlo . . . 36

10.3 Engine . . . 36

10.4 Web application . . . 36

10.5 Method . . . 37

A User manual **38**

A.1 Getting started . . . 38
A.2 Fetching and refreshing data . . . 38
A.3 Running a backtest . . . 38
A.4 Reading the results . . . 39
A.5 Strategies . . . 39
A.6 Building a strategy without code . . . 39
A.7 Writing a strategy in C# . . . 39
A.8 Comparing strategies . . . 39
A.9 Optimising parameters . . . 39
A.10 Train/test validation . . . 39
A.11 Monte Carlo . . . 40
A.12 Data quality and API quota . . . 40
A.13 Exporting reports . . . 40
A.14 Troubleshooting . . . 40

Chapter 1

Introduction

1.1 Objectives

The goal of this project was to build a backtesting website in C# and .NET. A backtest answers a simple question: if I had followed this trading rule in the past, what would have happened to my money? The subject asked us to fetch historical prices from the Alpha Vantage API, keep them locally to limit API calls, implement at least six trading strategies, simulate them on daily and weekly data, compute performance metrics such as the Sharpe ratio and the maximum drawdown, show the results in a web interface, and generate reports with graphs and trade logs.

We were a team of four, so one of our first concerns was to be able to work at the same time without blocking each other. Most of the architecture described in this report comes from that constraint.

1.2 Scope and summary of features

The application covers every requirement of the subject. Table 1.1 links each one to the place where we implemented it.

Requirement of the subject	**In QuantBacktest**
Fetch data from the free Alpha Vantage API	Typed HTTP client, daily and weekly series (chapter 3)
Store data locally	SQLite cache plus committed seed files
At least 6 strategies	10 strategies and a buy & hold baseline (chapter 4)
Custom strategies from templates	A C# template and a no-code rule builder
Simulation engine, daily and weekly	Next-open execution model (chapter 5)
Return, volatility, drawdown, Sharpe	13 metrics, computed for the strategy and for buy & hold
Web interface with charts and tables	ASP.NET Core MVC application (chapter 7)
Reports with graphs and trade logs	PDF report and CSV export (chapter 8)

Table 1.1: Requirements of the subject and where they are met.

Beyond the subject, we chose ten extra features that make the results more trustworthy, or simply easier to use:

1. advanced orders: stop-loss, take-profit, trailing stop and slippage;

2. an optimisation heatmap over two parameters, with an overfitting warning;
3. train/test validation (in-sample and out-of-sample);
4. a comparator of several strategies;
5. extra metrics: Sortino, Calmar, profit factor and exposure;
6. indicators drawn on the charts, and an explanatory card for each strategy;
7. a Monte Carlo simulation on the trades;
8. a data quality panel and an API quota counter;
9. a permanent, reproducible link for every result;
10. a Docker image, started with `docker compose up`.

The figures in this report were produced by the application itself, with the real services and the data shipped in the repository, unless a caption says otherwise.

1.3 Organisation of the work

Each of us owned one part. Table 1.2 gives the split. The whole code is in one GitHub repository, and what happened in it between 13 October and 8 November 2026 is described in chapter 9.

Member	**Responsibility**
Alexandre Tung	Alpha Vantage client, SQLite cache, seed files, data quality, API quota, storage of no-code strategies, PDF and CSV reports, Docker
Victor Haegeman	Technical indicators, strategies, C# template, no-code rule engine, Monte Carlo
Eloi Dumas	Backtesting engine, advanced orders, metrics, optimisation heatmap
Alexis Launay-Schapman	Web application, comparator, train/test, permanent links

Table 1.2: Split of the work.

Chapter 2

Architecture

2.1 Layers

The solution contains one shared project, `QuantBacktest.Core`, and one project per responsibility. Figure 2.1 shows the dependencies. They all point to `Core`.

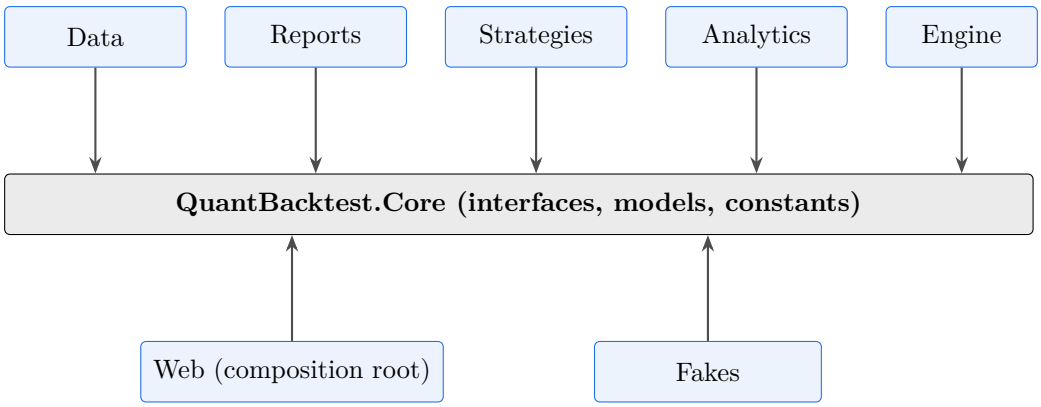


Figure 2.1: Project dependencies. Every project depends on `Core` only. `Web` also references the others, but only to plug them in.

2.2 The contract

`Core` contains interfaces (`IMarketDataService`, `IStrategy`, `IBacktestEngine`...), data models (`PriceSeries`, `BacktestResult`...) and constants, and nothing else. A method signature cannot say everything, though. For example, it cannot say at which price an order is filled. So we also wrote a document, `docs/CONTRACT.md`, with numbered rules: X1 to X15 for the execution of orders, M1 to M14 for the metrics, D1 to D8 for the data, and so on. The code, the tests and this report all quote these identifiers.

Once the contract was validated, we froze it. Changing it required an issue, the agreement of the four members and three approvals. In the end nobody had to change it.

2.3 The golden rule: no project knows another

A project never references the project of another part, only `Core`. The parts communicate through the interfaces, which are injected by dependency injection. Only `Web` references everything, because

it is the composition root: it calls `AddQuantBacktestData`, `AddQuantBacktestEngine` and the others at startup. As a result, each part can be tested and replaced independently, and all services are registered as singletons.

2.4 Request flow

Figure 2.2 shows what happens when a user runs a backtest. The web layer sends a `BacktestRequest`. The service validates it, gets the strategy from the catalog and the prices from the data service, and hands both to the engine. The engine returns a `BacktestResult`, which the web layer displays. The Monte Carlo service and the report generator take this same result as input.

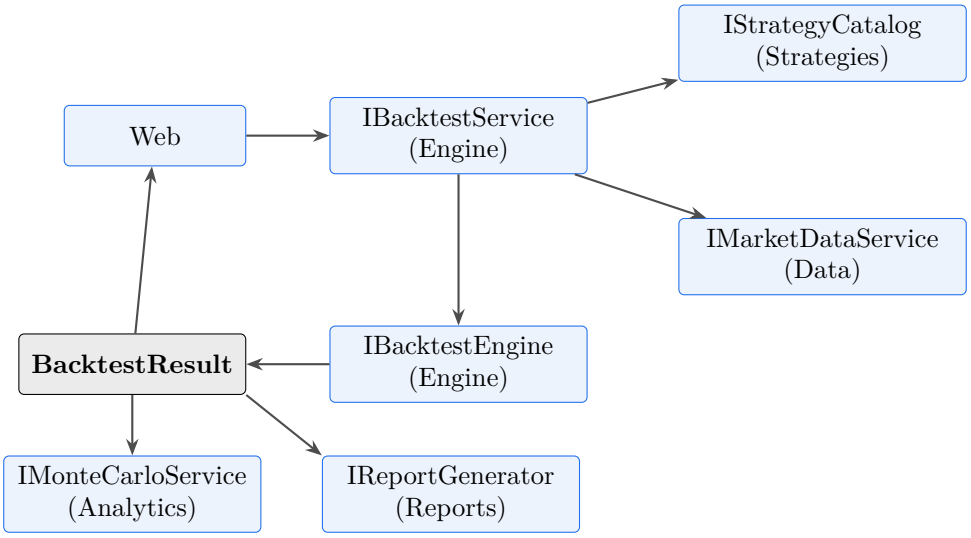


Figure 2.2: Flow of one backtest. Each box is an interface of `Core` and the project implementing it.

2.5 Fakes and parallel work

The `Fakes` project contains a plausible fake of every interface and a deterministic price generator. Because of it, the web pages could be built before the real engine existed. One configuration key per part (`QuantBacktest:UseFakes:Engine` and so on) chooses between the real service and the fake, so each real part could be plugged in as soon as it was merged. All five keys are now `false`. We kept one rule in mind: the fakes are only there to let others work. They do not define the correct behaviour; the contract does.

2.6 Conventions that avoided bugs

A few conventions, applied everywhere, removed whole families of bugs:

- prices, quantities and amounts are `decimal`, while returns and ratios are `double`;
- fractions in the code (0.12 means 12 %), percentages only on screen;
- `CultureInfo.InvariantCulture` whenever a number or a date is parsed or written outside the display;
- `DateOnly` for market dates, and an injected `TimeProvider` instead of `DateTime.Now`;

- an undefined ratio is `null` and displayed as “n/a”, never `NaN` and never replaced by 0.

Chapter 3

Data layer

3.1 What the free plan really gives

Before designing anything, we tested the endpoints with a free key. The result (Table 3.1, on MSFT, 15 October 2026) changed our plans: the full daily history and the adjusted daily series are premium features.

Endpoint	Result	Period	Bars
`TIME_SERIES_DAILY`, compact	Accepted	2026-05-22 to 2026-10-14	100
`TIME_SERIES_DAILY`, full	Refused (premium)		–
`TIME_SERIES_DAILY_ADJUSTED`	Refused (premium)		–
`TIME_SERIES_WEEKLY`	Accepted	1999-11-12 to 2026-10-14	1406
`TIME_SERIES_WEEKLY_ADJUSTED`	Accepted	1999-11-12 to 2026-10-14	1406

Table 3.1: Result of the endpoint test on the free plan.

We therefore use the compact daily endpoint and the adjusted weekly endpoint. Weekly data gives 27 years of prices adjusted for splits and dividends, so it is the natural choice for long tests. The adjustment applies the factor *adjusted close / close* to the four prices and leaves the volume unchanged. A series is marked as adjusted only if all of its bars are (rule D6). We did not add a third-party source for long daily histories, to avoid any licensing doubt.

3.2 Cache, seed files and the loading algorithm

Prices live in a local SQLite database, accessed with EF Core. It has four tables: `PriceBars`, `SeriesMetadata`, `ApiUsage` and `CustomStrategies`. Calling `GetSeriesAsync` follows these rules (D2, D3):

1. **Without refresh, cache not empty:** the cache is returned and no API call is made, so a backtest gives the same result every time.
2. **Without refresh, cache empty:** the seed file of the series is imported if it exists; otherwise the API is called.
3. **With refresh:** the API is called and the bars are merged by date. New dates are inserted, existing dates are replaced and no bar is ever deleted. If the call fails, the cache is left untouched.

The seed files in `data/seed/` are snapshots of the free endpoints for AAPL, MSFT, SPY and JPM, in daily and weekly. They allow the application to work without an API key, for example during a demonstration. Each series records its origin: `Seed`, `Api` or `Mixed`. Figure 3.1 shows the content of a database, and Figure 3.2 a real refresh from the application.

```
PS C:\Users\tunga> python "C:\Users\tunga\AppData\Local\Temp\claude\e6a9a34502162c4d\74497b4f-22cd-410b-afa9-49024e1acd51\scra
tchpad\show_tables.py"
Database: C:/Users/tunga/Downloads/QuantBacktest-DataDemo/App_Data/demo.db

Tables and row counts
  ApiUsage               1 rows
  CustomStrategies       0 rows
  PriceBars           1508 rows
  SeriesMetadata         2 rows

PriceBars (5 most recent MSFT daily rows)
  Symbol Timeframe Date        Open     High     Low      Close    Volume     Adjusted
  MSFT   Daily     2026-10-15  533.32   541.13   531.22   539.44   24151211   0
  MSFT   Daily     2026-10-14  545.27   545.75   532.54   533.69   22324295   0
  MSFT   Daily     2026-10-13  544.64   551.15   543.06   543.45   23851953   0
  MSFT   Daily     2026-10-12  537.53   542.74   535.70   541.15   18314943   0
  MSFT   Daily     2026-10-09  528.24   537.22   526.24   535.07   18500612   0

SeriesMetadata
   MSFT Daily Mixed 2026-10-16 09:48:08.8249678+00:00
   SPY Weekly Mixed 2026-10-16 09:48:11.4629681+00:00
PS C:\Users\tunga>
```

Figure 3.1: Tables of the SQLite cache and recent rows of `PriceBars`, read in a terminal (database of a console run of the data layer, same schema as the application).

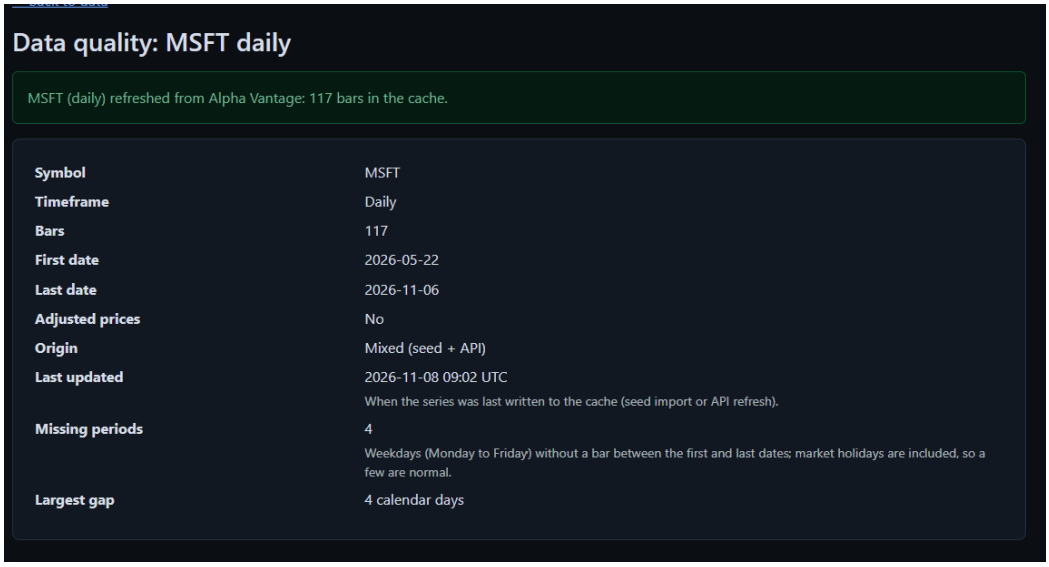


Figure 3.2: A real refresh of MSFT daily from the application: the API bars are merged into the seed series, which has 117 bars and the origin `Mixed`.

3.3 Talking to Alpha Vantage safely

The project guide gave us a good starting point: `GetAsync`, then parsing the CSV by splitting on commas. While working on it, we found three traps.

Errors arrive with HTTP 200. With an invalid key or when the limit is reached, Alpha Vantage answers with status 200 and a small JSON document, even when CSV was requested. A simple test on the status code would save this JSON as if it were data. Our client detects a body starting with `{` and classifies it into `SymbolNotFound`, `RateLimitExceeded` or `InvalidResponse`.

Decimal separators. `double.Parse("123.45")` without a culture misreads the number on a computer configured in French, which is the default on our machines. The parser always uses

`InvariantCulture`, and one test runs under `fr-FR` to prove it.

The API key can leak. The key is part of the URL, and .NET logs request URLs by default. Worse, the rate limit message of Alpha Vantage repeats the key. So the key is only read from user-secrets or an environment variable, our error messages are written by us and never copied from the API, and HTTP logging is removed:

```
services.AddHttpClient<AlphaVantageClient>(
        AlphaVantageClient.HttpClientName,
        client => client.Timeout = AlphaVantageClient.Timeout)
    .RemoveAllLoggers();
```

After a real refresh in the Docker container, we searched the whole log for the key, for `apikey=` and for the host name of the API, and found no occurrence of any of them.

3.4 Parsing, quota and concurrency

The parser accepts exactly two headers (raw and adjusted). A malformed line, or one that breaks the invariants of a bar (High $\geq \max(\text{Open}, \text{Close})$ and Low $\leq \min(\text{Open}, \text{Close})$), is dropped with a warning. We never try to repair it.

The free plan allows 25 requests per day. Every HTTP request is counted per UTC day just before it is sent, so a failed request counts too. The quota is checked first, then the key. Simultaneous calls for the same series share a single download, thanks to one lock per symbol and timeframe: ten simultaneous calls send one HTTP request.

3.5 Data quality and API quota

A backtest is only as good as its data, so we added a quality report, computed on cached data only (rule D7). It flags four kinds of anomalies:

- a **price jump**, when the close moves by more than 25 % in one bar;
- a **zero volume**;
- a **stale price**, at the fifth identical close in a row;
- a **gap**, when more than 5 calendar days (daily) or 10 days (weekly) separate two bars.

It also counts the missing periods and the largest gap. Figure 3.3 shows that it finds real events: the 11-day gap of September 2001, when US markets closed after the attacks of 11 September, and the violent moves of JP Morgan during the 2008 crisis. These are true market moves, so a flag is an invitation to look, not an error.

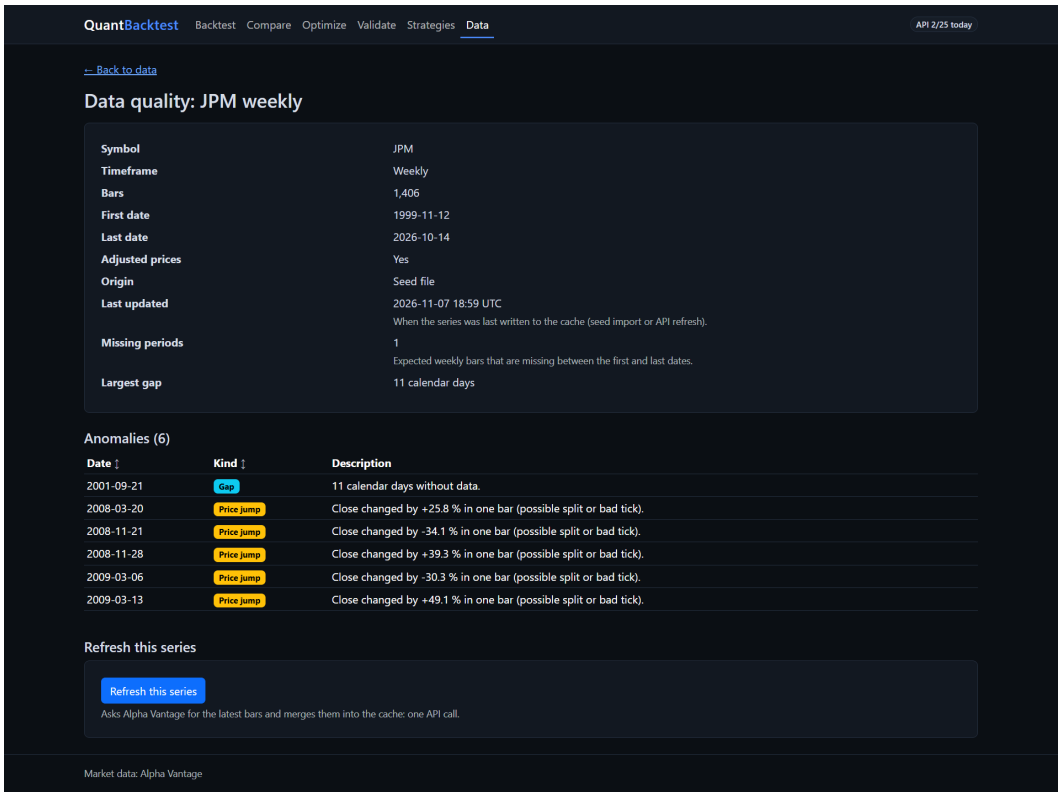


Figure 3.3: Quality report of JPM weekly: one gap and five price jumps.

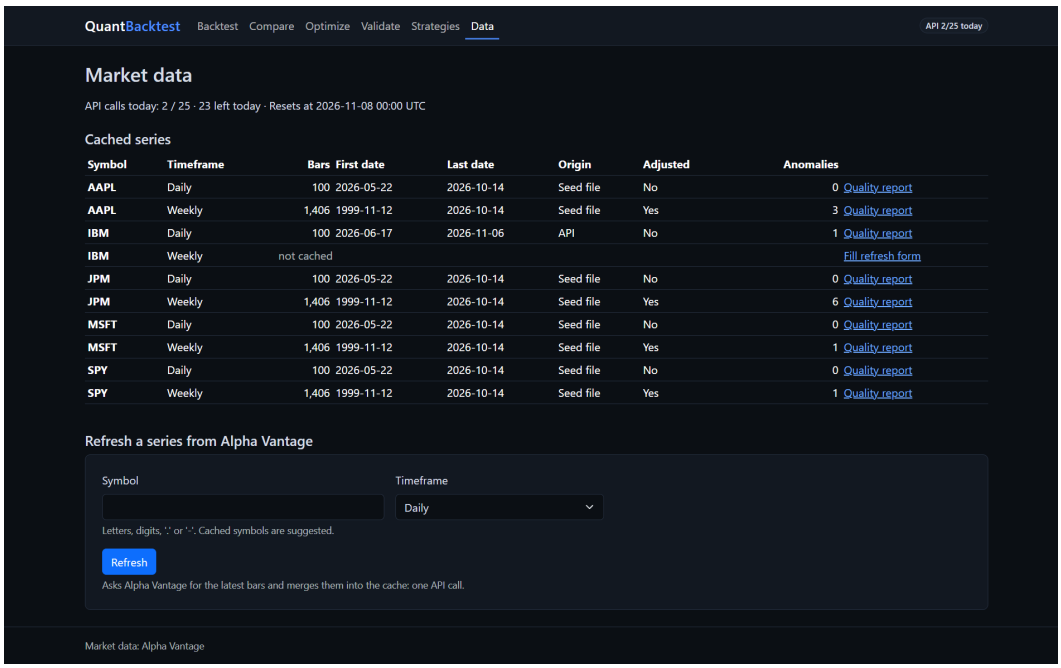


Figure 3.4: The Data page: cached series with their origin, adjustment and anomalies, the API quota and the refresh form.

Chapter 4

Strategies

4.1 What a strategy is allowed to do

In our design, a strategy only gives an opinion. For each bar it returns `Buy`, `Sell` or `Hold`. It knows nothing about capital, fees or stops, which belong to the engine. The common rules are strict:

- a strategy is pure and deterministic: no state, no network, no clock, no randomness (S1);
- it returns exactly one signal per bar, and `Hold` while its indicators warm up (S2);
- the signal of bar i may only depend on bars 0 to i: no look-ahead (S3);
- if the buy and sell conditions are both true on a bar, the signal is `Sell` (S4).

4.2 Indicators

All strategies share a single implementation of the indicators, and the no-code engine uses it too, so two strategies cannot disagree on what "RSI 14" means.

Indicator	**Definition**
SMA(n)	Average of the last n values
EMA(n)	$\alpha = 2/(n+1)$, started from the SMA of the first n values
Bollinger(n, k)	SMA(n) $\pm k\times$ population standard deviation over n
RSI(n)	Wilder's smoothing of gains and losses, $100 - 100/(1 + \text{gains/losses})$
ROC(n)	$x_i/x_{i-n} - 1$
MACD(f, s, g)	EMA(f) − EMA(s), and an EMA(g) of this line as signal
Highest high / lowest low(n)	Over the n previous bars, the current bar being excluded

Table 4.1: Technical indicators (rules I1 to I9).

4.3 The built-in strategies

The catalog contains ten strategies and a buy & hold baseline. Classes are discovered by reflection, so adding a strategy does not require touching the web project.

Strategy	**Family**	**Rule (default parameters)**
Moving average crossover	Trend	Buy when SMA(20) crosses above SMA(50), sell when it crosses below
MACD crossover	Trend	Buy when the MACD$(12, 26, 9)$ line crosses above its signal, sell when it crosses below
Momentum (ROC)	Momentum	Buy when ROC(14) crosses above the threshold (0), sell when it crosses below
Donchian breakout	Breakout	Buy when the close exceeds the highest high of 20 bars, sell below the lowest low of 10 bars
Bollinger reversion	Reversion	Buy when the close falls below the lower band $(20, 2)$, sell when it crosses above the middle band
RSI reversion	Reversion	Buy when RSI(14) < 30, sell when RSI(14) > 70
Bollinger + RSI	Reversion	Buy when the close is below the lower band and RSI < 30; sell above the middle band or when RSI > 70
Trend-filtered RSI-2	Reversion	Buy when the close is above SMA(200) and RSI(2) < 10, sell when the close is above SMA(5)
Consecutive down days	Reversion	Buy after 3 consecutive falling closes, sell at the first rising close
Turn of the month	Calendar	Buy 1 business day before the end of the month, sell on the 3rd bar of the next month (daily only)
Buy and hold	Baseline	Buy and keep

Table 4.2: Built-in strategies.

Each strategy comes with a card written for the user: the idea, the exact entry and exit rules, and when the strategy loses money (Figure 4.1).

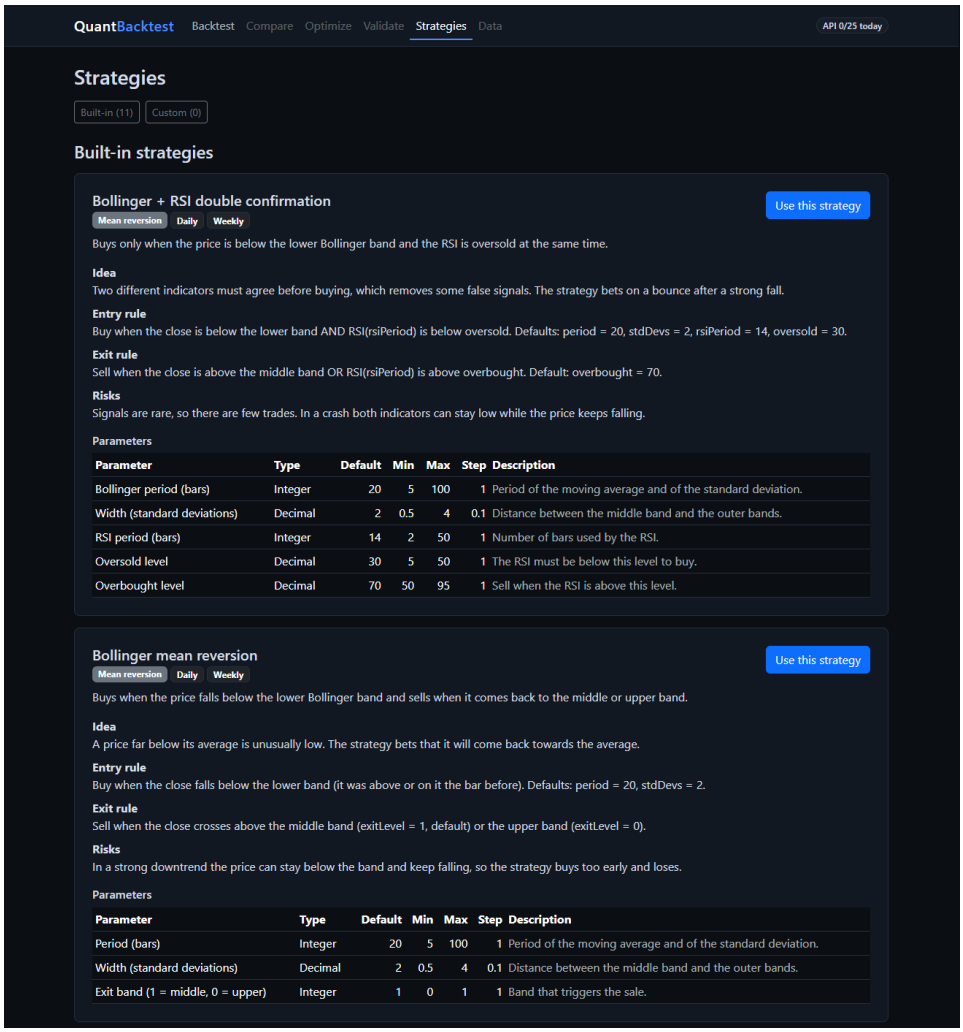


Figure 4.1: First two cards of the strategy catalog, generated from the strategies themselves.

4.4 A first look at the results

To see how the strategies behave, we ran each of them on SPY weekly, with default parameters, a capital of 10,000 and a fee of 0.1 % per fill. Table 4.3 gives the result, on 1406 bars from November 1999 to October 2026. *Turn of the month* only accepts daily data, so it is absent.

Strategy	Cum.	Ann.	Vol.	Sharpe	Max DD	Trades	Exposure
Moving average crossover	+759.4 %	8.32 %	12.4 %	0.70	31.8 %	9	70.9 %
Donchian breakout	+300.9 %	5.29 %	9.8 %	0.57	27.9 %	22	61.0 %
Buy and hold (strategy)	+781.5 %	8.42 %	17.6 %	0.55	54.6 %	1	99.9 %
Momentum (ROC)	+258.5 %	4.86 %	11.1 %	0.48	38.5 %	68	68.7 %
Trend-filtered RSI-2	+111.7 %	2.83 %	7.7 %	0.40	23.5 %	42	9.2 %
RSI reversion	+156.9 %	3.57 %	11.1 %	0.37	43.4 %	4	20.6 %
MACD crossover	+94.0 %	2.49 %	10.2 %	0.29	40.9 %	59	53.3 %
Bollinger + RSI	+56.6 %	1.68 %	8.6 %	0.24	37.4 %	6	5.0 %
Bollinger reversion	+67.7 %	1.94 %	11.2 %	0.23	41.4 %	20	14.2 %
Consecutive down days	+30.0 %	0.98 %	7.2 %	0.17	39.8 %	57	7.3 %

Table 4.3: Strategies on SPY weekly, default parameters, sorted by Sharpe ratio.

Three things are worth noting. First, only the simple moving average crossover comes close to buy & hold in return. It does so with a much smaller drawdown (31.8 % against 54.6 %), because it stays out of the market during long declines. Second, the mean reversion strategies are rarely invested on weekly bars (between 5 % and 21 % of the time), which explains their low returns: they cannot earn anything while they stay out. Third, several of these results rest on very few trades (9 for the crossover, 4 for RSI reversion), so we would not conclude that one strategy is "better" from this table alone. Chapter 6 deals with this problem.

One detail deserves an explanation: the buy & hold *strategy* returns +781.5 % while the buy & hold *benchmark* returns +800.7 %. It is not a bug. The benchmark buys at the Open of the first bar (rule X14), whereas a strategy's signal at the close of the first bar is only executed at the Open of the second bar (rule X4). The strategy also pays a fee to exit at the end of the period, while the benchmark is never sold.

4.5 Writing a strategy in C#

The file `Custom/TemplateStrategy.cs` is a fully commented template, hidden from the catalog by the `[HiddenStrategy]` attribute. To add a strategy, a developer copies it, removes the attribute, changes the identifier, and writes the documentation, the parameters and the rules. The strategy then appears in every form of the website without any other change.

4.6 Building a strategy without code

The subject asks that users can define their own strategies. Writing C# is not a realistic option for every user, so we added a rule builder (Figure 4.2). A strategy is made of two rules, entry and exit, each with 1 to 5 conditions combined by "all" or "any". A condition compares two operands with `>`, `<`, "crosses above" or "crosses below". An operand can be a price, the volume, a constant, or an indicator with its period.

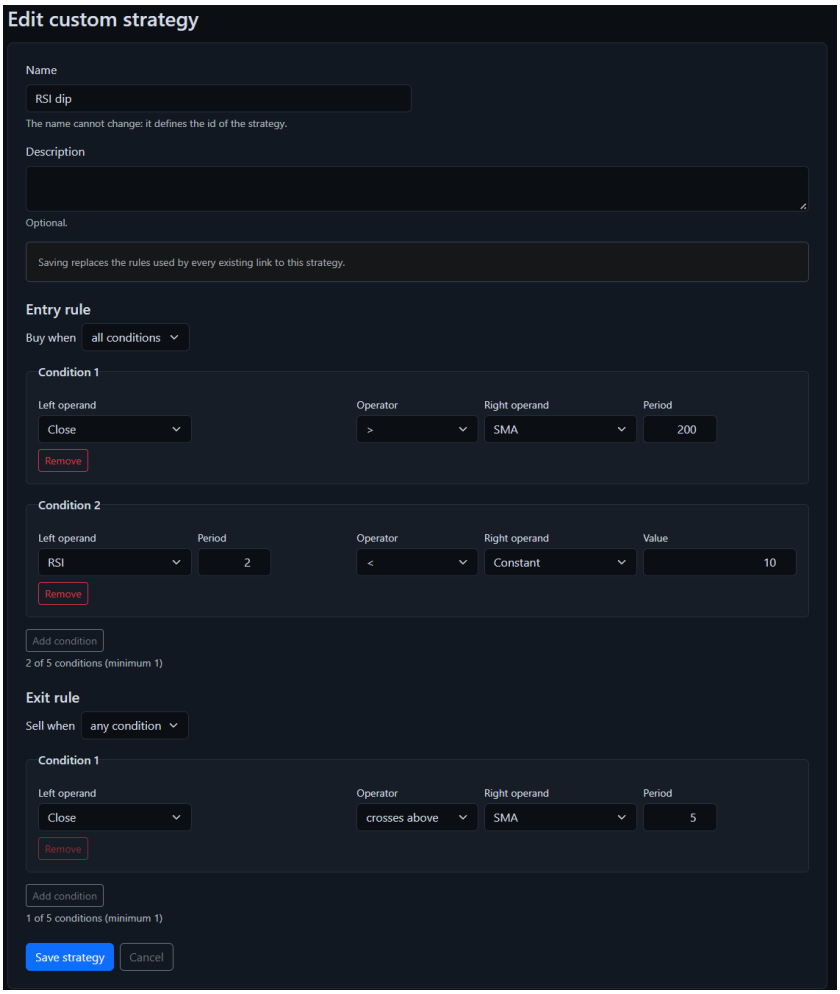


Figure 4.2: The no-code rule builder, editing the strategy "RSI dip".

`RuleStrategyFactory` validates a definition and returns *all* its problems at once, each with its position (for example "Entry condition 2: the right operand needs a period between 1 and 500."). A valid definition becomes a normal `IStrategy`. Its documentation is generated from the rules, for example "Buy when ALL of: Close > SMA(200); RSI(2) < 10". The definition itself is stored as JSON in SQLite. A condition is false as long as one of its values is undefined, so warm-up periods need no special handling.

Chapter 5

Backtesting engine

5.1 Execution model

The most common mistake in a backtest is to use information that was not yet available. Our model is deliberately simple. A strategy gives its signal at the close of a bar, and the engine executes it at the **Open of the next bar** (X4). A signal therefore cannot use a price that did not exist yet. Positions are long only, without leverage, and use 100 % of the cash (X5).

With a price p, a slippage s and a fee rate f, rule X6 gives:

$$\text{fill}_{\text{buy}} = p\,(1+s), \qquad \text{fill}_{\text{sell}} = p\,(1-s),$$

$$q = \frac{\text{cash}}{\text{fill}_{\text{buy}}\,(1+f)}, \qquad \text{proceeds} = q\,\text{fill}_{\text{sell}}\,(1-f).$$

For every bar i of the period, the engine applies these steps in this order (X7):

1. execute at the Open the signal of bar $i-1$;
2. if a position is open, check the stop-loss, trailing stop and take-profit;
3. on the last bar, sell any open position at the Close (`EndOfData`);
4. record the equity, cash $+\, q \times$ Close.

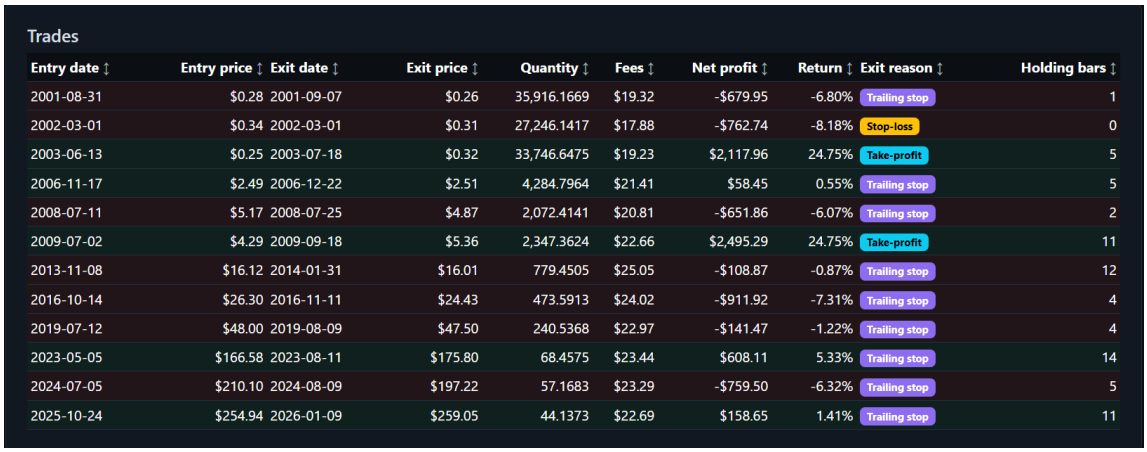
Bars before the requested period are still given to the strategy, so that its indicators are warmed up, but their signals are ignored (X2).

5.2 Advanced orders

Let E be the entry fill and *peak* the highest High since the entry, before the current bar. The stop level is $S = \max\big(E(1-\text{SL}),\ \text{peak}(1-\text{TS})\big)$ and the target is $T = E(1+\text{TP})$. The engine then applies rule X8:

- **Gap** (not on the entry bar): if the bar opens at or below S, or at or above T, the exit is at the Open, because the level could not be reached.
- **During the bar:** if Low $\le S$ the exit is at S; otherwise, if High $\ge T$, the exit is at T.

When both levels are touched in one bar, we cannot know which came first with daily or weekly data. We chose the prudent assumption: the stop always comes first. Figure 5.1 shows these exits on a real backtest.



Trades

Entry date	Entry price	Exit date	Exit price	Quantity	Fees	Net profit	Return	Exit reason	Holding bars
2001-08-31	$0.28	2001-09-07	$0.26	35,916.1669	$19.32	-$679.95	-6.80%	Trailing stop	1
2002-03-01	$0.34	2002-03-01	$0.31	27,246.1417	$17.88	-$762.74	-8.18%	Stop-loss	0
2003-06-13	$0.25	2003-07-18	$0.32	33,746.6475	$19.23	$2,117.96	24.75%	Take-profit	5
2006-11-17	$2.49	2006-12-22	$2.51	4,284.7964	$21.41	$58.45	0.55%	Trailing stop	5
2008-07-11	$5.17	2008-07-25	$4.87	2,072.4141	$20.81	-$651.86	-6.07%	Trailing stop	2
2009-07-02	$4.29	2009-09-18	$5.36	2,347.3624	$22.66	$2,495.29	24.75%	Take-profit	11
2013-11-08	$16.12	2014-01-31	$16.01	779.4505	$25.05	-$108.87	-0.87%	Trailing stop	12
2016-10-14	$26.30	2016-11-11	$24.43	473.5913	$24.02	-$911.92	-7.31%	Trailing stop	4
2019-07-12	$48.00	2019-08-09	$47.50	240.5368	$22.97	-$141.47	-1.22%	Trailing stop	4
2023-05-05	$166.58	2023-08-11	$175.80	68.4575	$23.44	$608.11	5.33%	Trailing stop	14
2024-07-05	$210.10	2024-08-09	$197.22	57.1683	$23.29	-$759.50	-6.32%	Trailing stop	5
2025-10-24	$254.94	2026-01-09	$259.05	44.1373	$22.69	$158.65	1.41%	Trailing stop	11

Figure 5.1: Trade log of a moving average crossover on AAPL weekly with an 8 % stop-loss, a 25 % take-profit and a 10 % trailing stop.

5.3 Metrics

With P the number of periods per year (252 or 52), E_t the equity, $r_t = E_t/E_{t-1} - 1$ and r_f the risk-free rate per period:

Metric	Formula
Cumulative return	$E_{\text{final}}/I - 1$, with I the initial capital
Annualized return	$(E_{\text{final}}/I)^{1/\text{years}} - 1$, with years = days/365.25
Volatility	sample standard deviation of r_t, times $\sqrt{P}$
Sharpe ratio	$\dfrac{\overline{r} - r_f}{\sigma(r)}\sqrt{P}$
Sortino ratio	$\dfrac{\overline{r} - r_f}{DD}\sqrt{P}$, with $DD = \sqrt{\frac{1}{n}\sum \min(0, r_t - r_f)^2}$
Maximum drawdown	largest $(\text{peak} - E_t)/\text{peak}$
Calmar ratio	annualized return / maximum drawdown
Win rate	share of trades with a positive net profit
Profit factor	sum of gains / absolute sum of losses
Exposure	share of the points where a position is held

Table 5.1: Main performance metrics (rules M1 to M12). The benchmark uses the same formulas (M13).

A ratio that cannot be defined, such as the Sharpe ratio of a flat curve, is `null` and displayed as "n/a". This matters more than it seems: replacing it with 0 would present an undefined value as a real result.

5.4 Validation of requests and errors

`RequestValidator` checks every field and names the field and the allowed range in its message. A user mistake raises a `BacktestValidationException`, shown above the form with the input kept (Figure 5.2). A wrong strategy, for example one returning the wrong number of signals, raises an `InvalidOperationException`, because it is a bug. Keeping these two cases apart means that a message shown to the user is always something the user can fix.

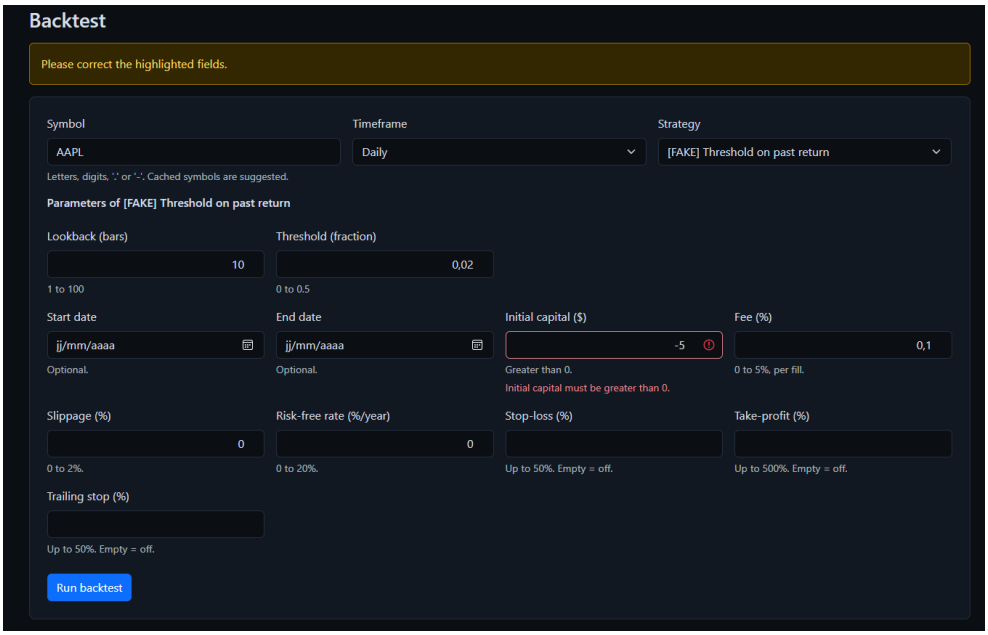


Figure 5.2: Validation of a backtest form: an initial capital of -5 is refused (screenshot taken with the fake services, during development).

5.5 Optimisation heatmap

The heatmap tests every pair of values of two parameters, up to 625 cells. The prices are loaded only once, then each cell goes through the same validation and the same engine as a normal backtest. An invalid pair (for example `fast` $\geq$ `slow`) gives an empty cell with its message, and does not stop the grid.

An optimiser will always find a best cell, even on noise. So we warn the user: if the best value is positive and the average of its neighbours is below half of it (rule O4), the page shows an "isolated peak" banner. In Figure 5.3, the best profit factor is 110.28, a suspicious number which comes from very few trades, and the banner appears. The colour of a cell shows its rank, because a linear scale was unreadable on real grids.

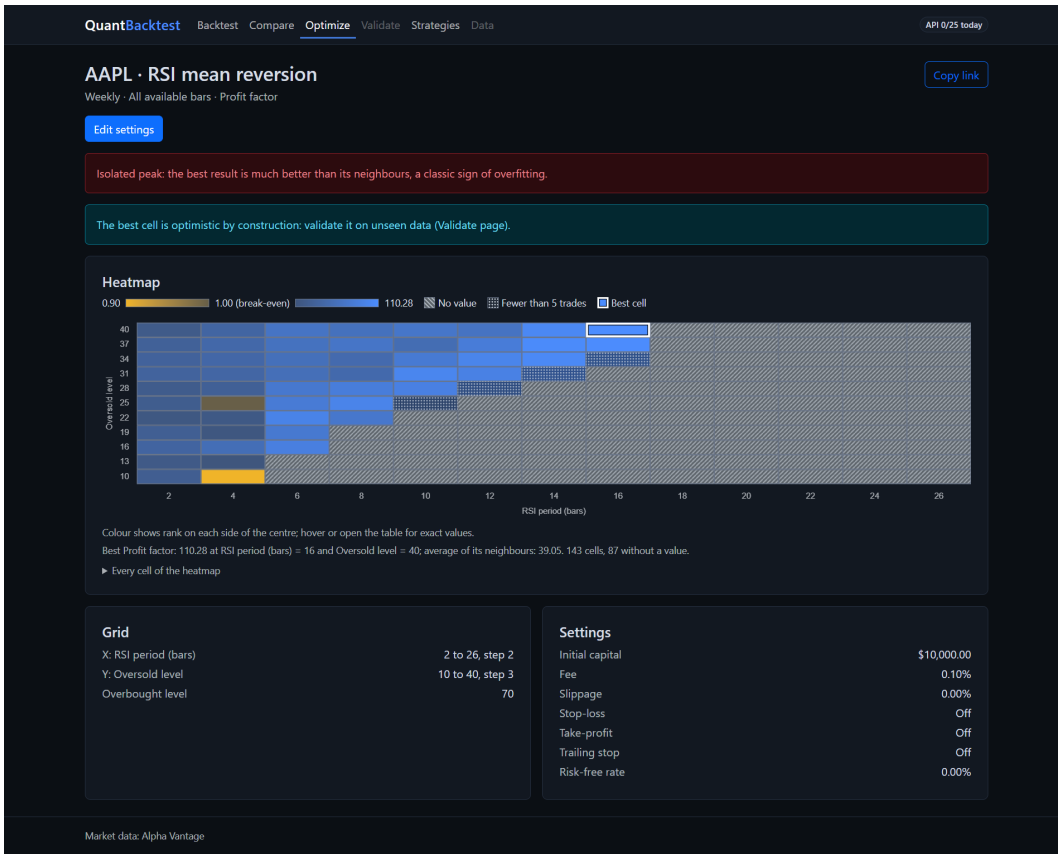


Figure 5.3: Heatmap of RSI mean reversion on AAPL weekly, with the isolated peak warning.

5.6 How we checked the numbers

The engine has 184 tests. Their expected values are recomputed in the test from the formula, never copied from the output of the engine. Scenarios on six bars were also computed independently in a spreadsheet (Figure 5.4). Finally, property tests run random signals on synthetic series, and check invariants: the equity is never negative, the final equity equals the capital plus the sum of the profits, trades never overlap, and truncating the future does not change the past.

Scenario S1 - signals B H S H H H - hand check (capital 10 000, fee 0.1 %, no slippage)			
To compare with the engine tests: quantity 97.94, net profit 371.39, fees 20.37, equity 10087.95 / 10283.83 / 10371.39			
Capital	10000,		input
Fee rate	0,001		input (0.1 %)
Entry price (Open of bar 1)	102,		input: Buy signal on bar 0, executed at the next Open
Exit price (Open of bar 3)	106,		input: Sell signal on bar 2, executed at the next Open
Quantity	97,9413		capital / (entry x (1 + fee)) [rule X6]
Cost of the purchase	10000,0000		q x entry x (1 + fee)
Proceeds of the sale	10371,3933		q x exit x (1 - fee) [rule X6]
Fees (entry + exit)	20,3718		q x entry x fee + q x exit x fee
Net profit	371,3933		proceeds - cost [rule X13]
Return of the trade	0,0371		net profit / cost
Bar	**Close**	**Equity**	**How it is computed**
Bar 0	101	10000,0000	first point = capital [rule X11]
Bar 1	103	10087,9513	bought at the Open of bar 1, valued at the Close: q x Close
Bar 2	105	10283,8338	q x Close
Bar 3	104	10371,3933	sold at the Open of bar 3: everything is cash
Bar 4	108	10371,3933	cash
Bar 5	110	10371,3933	cash

Figure 5.4: Scenario S1 recomputed by hand: quantity, fees, net profit and equity per bar.

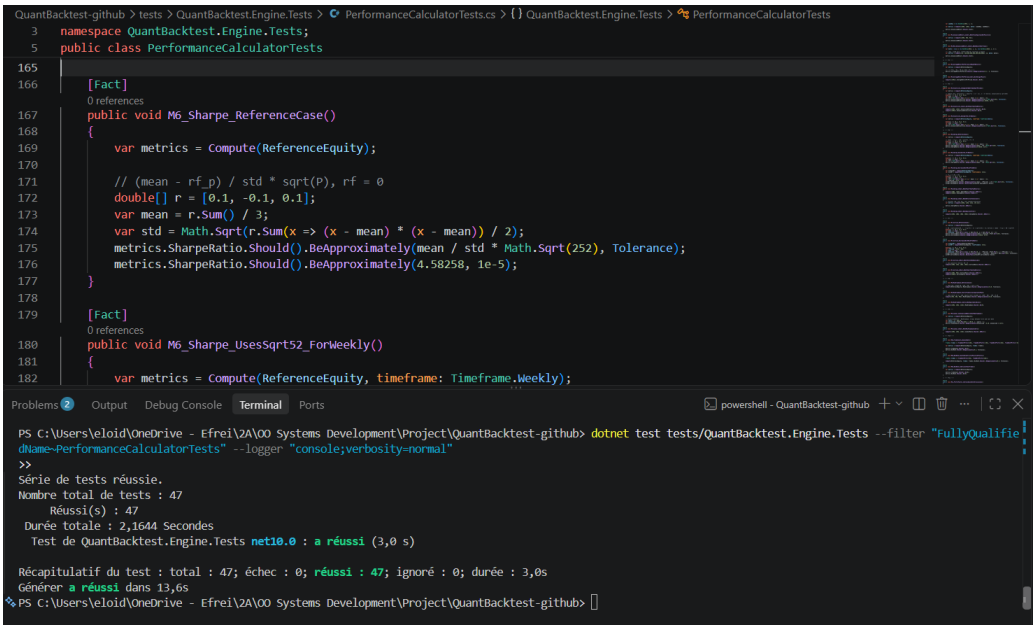


Figure 5.5: A test of the Sharpe ratio, with the formula written in the test, and the 47 metric tests passing.

Chapter 6

Robustness analysis

A good backtest proves little on its own: the result can come from luck or from fitting the past. We added two tools to measure this.

6.1 Train/test validation

`/Validate` splits the selected period in two (rules T1 to T3). The first $\lfloor N \times \text{train} \rfloor$ bars, with a default of 70 %, form the training period. The page runs the heatmap on them only, takes the best cell, and tests those parameters once on the remaining bars. The test period may use earlier bars to warm up its indicators, which is not an information leak since every signal still depends only on past bars.

Figure 6.1 shows an example on AAPL weekly since 2010, optimised on the Sharpe ratio. The 876 bars split into 613 training bars and 263 test bars. The best parameters are fast 26 and slow 35. The Sharpe ratio falls from 1.02 in training to 0.75 in test. This drop is expected, since the best cell was chosen because it was the best on those data. With 5 and 6 trades, neither number is statistically solid.

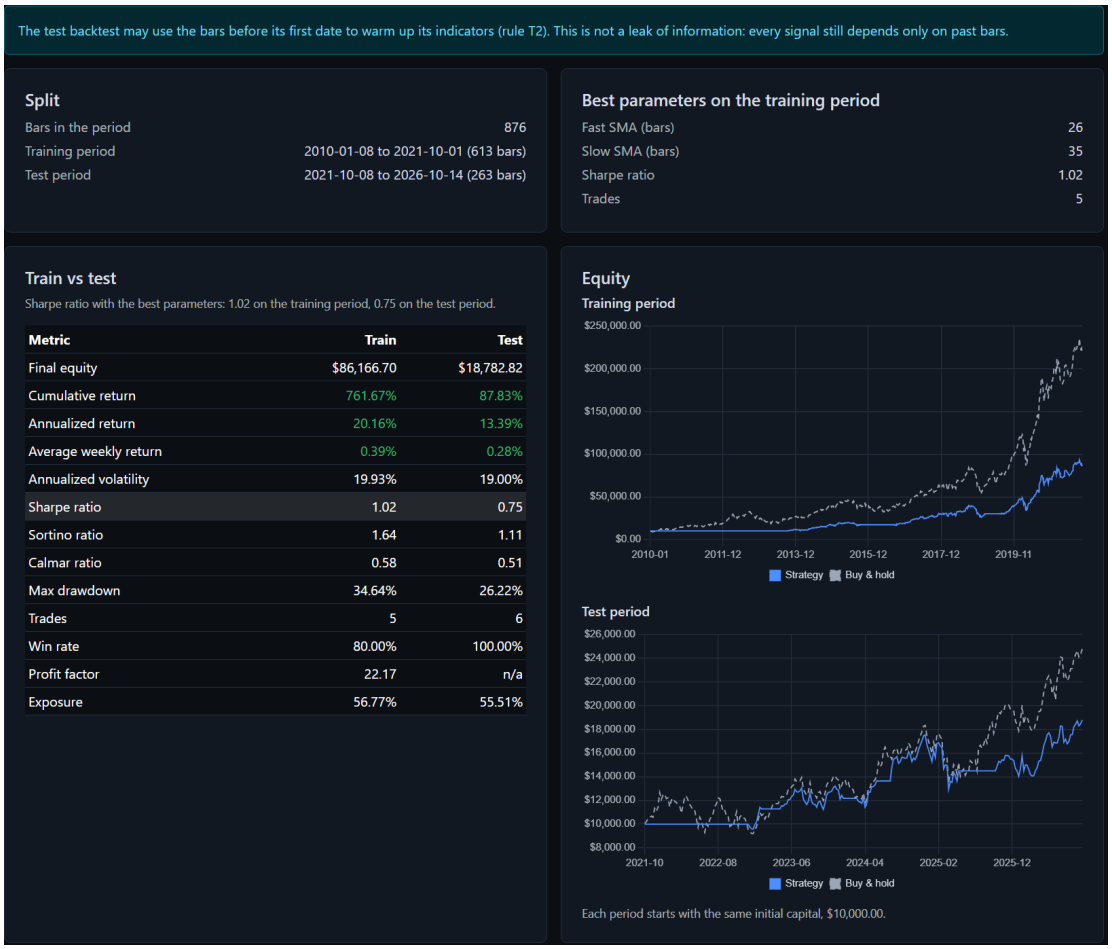


Figure 6.1: Train/test validation of the moving average crossover on AAPL weekly.

6.2 Monte Carlo simulation

The Monte Carlo analysis asks how much of a result depends on the particular sequence of trades. We take the returns r_j of the k trades of a backtest (at least 5), and we draw k of them at random with replacement. The trajectory starts at $V_0 = 1$ and compounds: $V_j = V_{j-1}(1 + r)$. This gives a final return and a maximum drawdown. Repeating it between 100 and 10,000 times gives two distributions, summarised by their percentiles and histograms (rules MC1 to MC6). All draws use one generator seeded by the user, so the same seed always gives the same result.

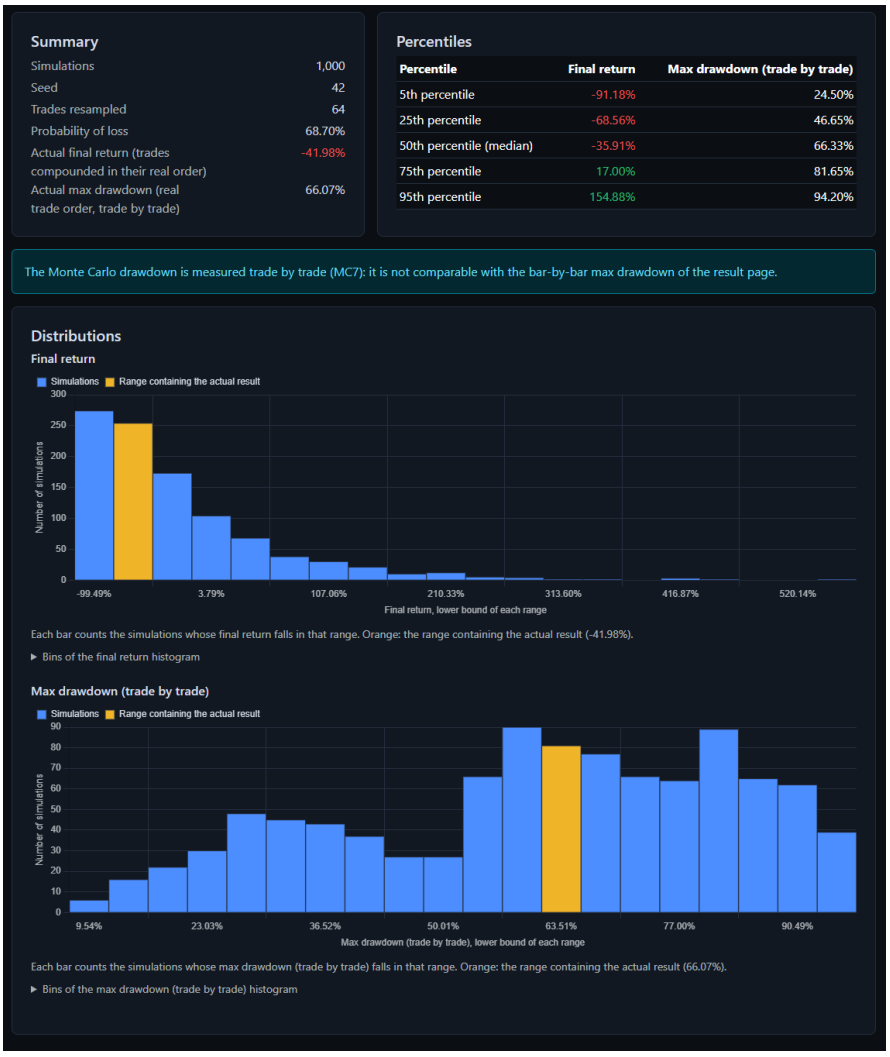


Figure 6.2: Monte Carlo of *Consecutive down days* on AAPL weekly (64 trades, 1,000 simulations).

In Figure 6.2, the strategy has a 68.7 % probability of loss, and its actual result (-41.98 %) is in the middle of the distribution: it is simply a losing strategy, and bad luck does not explain it. On the contrary, the moving average crossover on MSFT weekly in Figure 8.4 has 8.5 % probability of loss, but the percentiles go from -17.4 % to +29,111 %. With 15 trades, one of which earned +521 %, the result depends strongly on how many times this single trade is drawn. Looking only at the final figure would have hidden that.

Note that this drawdown is measured trade by trade (rule MC7). It is not comparable with the bar-by-bar maximum drawdown of the results page, and the interface and the PDF both say so.

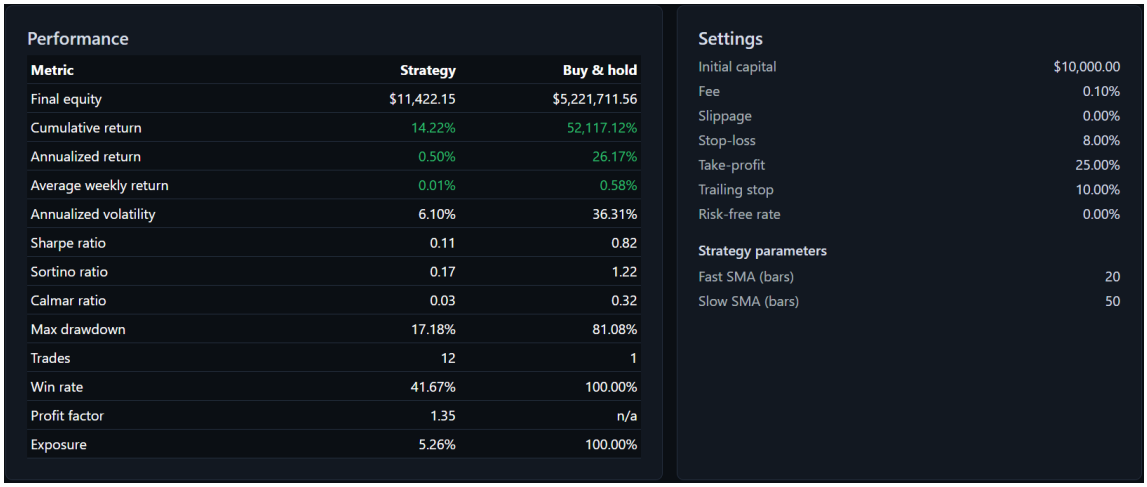
Chapter 7

Web application

7.1 Pages and user journey

The application uses ASP.NET Core MVC. The navigation has six entries: Backtest, Compare, Optimize, Validate, Strategies and Data. A typical session starts on the Backtest page, where the user chooses a symbol, a timeframe, a strategy and execution settings. The parameter fields are generated from the definition of the chosen strategy, so no strategy is hard-coded in the web project.

The results page (Figures 7.1 and 7.2) shows the 13 metrics against buy & hold, four charts drawn with Chart.js and a sortable trade log. From there, the user can download the PDF or the trades, run a Monte Carlo, optimise the strategy, or copy a permanent link.



Performance

Metric	Strategy	Buy & hold
Final equity	$11,422.15	$5,221,711.56
Cumulative return	14.22%	52,117.12%
Annualized return	0.50%	26.17%
Average weekly return	0.01%	0.58%
Annualized volatility	6.10%	36.31%
Sharpe ratio	0.11	0.82
Sortino ratio	0.17	1.22
Calmar ratio	0.03	0.32
Max drawdown	17.18%	81.08%
Trades	12	1
Win rate	41.67%	100.00%
Profit factor	1.35	n/a
Exposure	5.26%	100.00%

Settings

Initial capital	$10,000.00
Fee	0.10%
Slippage	0.00%
Stop-loss	8.00%
Take-profit	25.00%
Trailing stop	10.00%
Risk-free rate	0.00%

Strategy parameters

Fast SMA (bars)	20
Slow SMA (bars)	50

Figure 7.1: Results page of a real backtest: AAPL weekly, moving average crossover with stops.

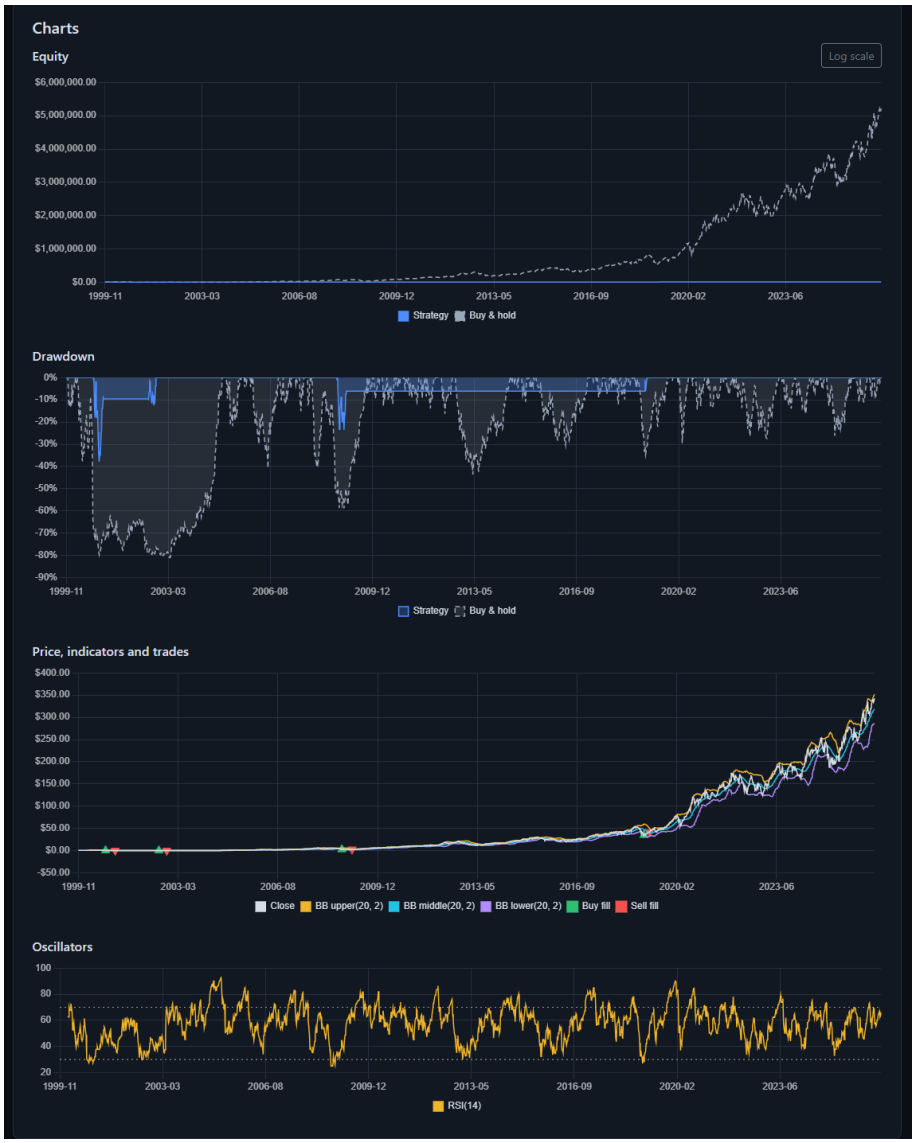


Figure 7.2: Charts of a result: equity, drawdowns, price with indicators and trade markers, and an oscillator.

7.2 Design choices

- **No financial logic in the web project.** Controllers call the services and view models format the values. The only computations are for presentation, plus the train/test split.
- **Only the contract exceptions are caught.** `BacktestValidationException` and `MarketDataException` become an alert above the form, with the input kept. Anything else is a bug and reaches a generic error page.
- **Forms that work without JavaScript.** The server renders the forms and validates them; JavaScript modules only improve them.
- **Local libraries.** Bootstrap, Chart.js and the matrix plugin are served by the application, with no CDN.
- **Colours with a meaning.** Green and red are reserved for gains and losses.

The interface also works on small screens, down to 360 pixels wide (Figure 7.4).

7.3 Comparator

`/Compare` runs 2 to 5 strategies with their default parameters on the same symbol, period and settings. It draws one equity curve per strategy, plus buy & hold, and a sortable table of metrics (Figure 7.3). If one strategy fails, its message is displayed without blocking the others.



Strategy	Cumulative return	Annualized return	Annualized volatility	Sharpe ratio	Sortino ratio	Calmar ratio	Max drawdown	Trades	Profit factor	Exposure
Momentum (rate of change)	149.16%	8.06%	21.07%	0.47	0.67	0.21	38.24%	31	2.08	66.72%
Donchian channel breakout	273.89%	11.84%	17.96%	0.71	1.12	0.53	22.49%	12	6.06	56.49%
MACD crossover	288.68%	12.21%	17.93%	0.73	1.15	0.47	26.09%	26	3.35	52.27%
Buy & hold	1,261.88%	24.82%	27.41%	0.96	1.47	0.72	34.64%	1	n/a	100.00%

Figure 7.3: Comparator on AAPL weekly since 2015: three strategies and buy & hold. On this symbol, buy & hold returned +1,261.88 %, far more than any strategy.

7.4 Permanent links

Every result has a `GET` URL containing all its settings (symbol, timeframe, strategy, dates, capital, fee, slippage, stops, risk-free rate and parameters). One class, `BacktestQuery`, reads and writes these URLs. It writes one canonical form and redirects to it once, so two equivalent links become the same link. Since the cache is not refreshed automatically, copying a URL gives the same result later, as long as nobody refreshes that series.

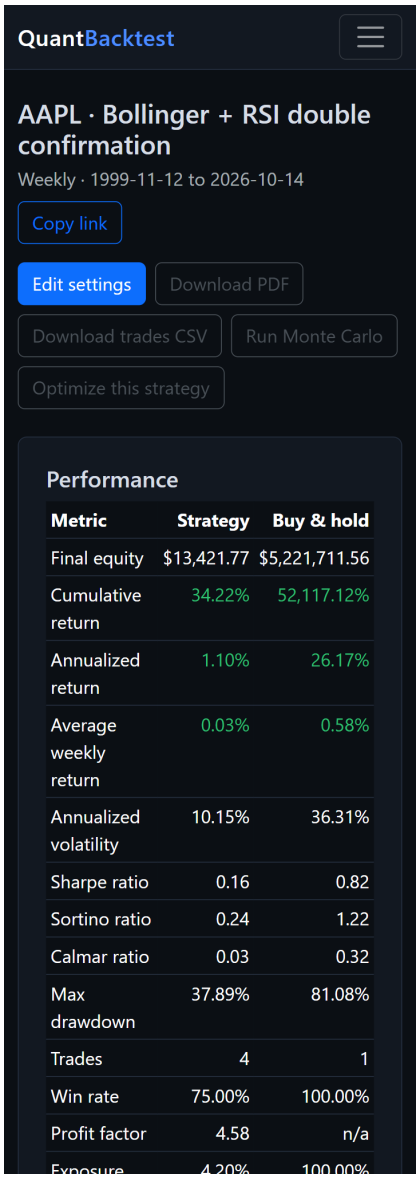


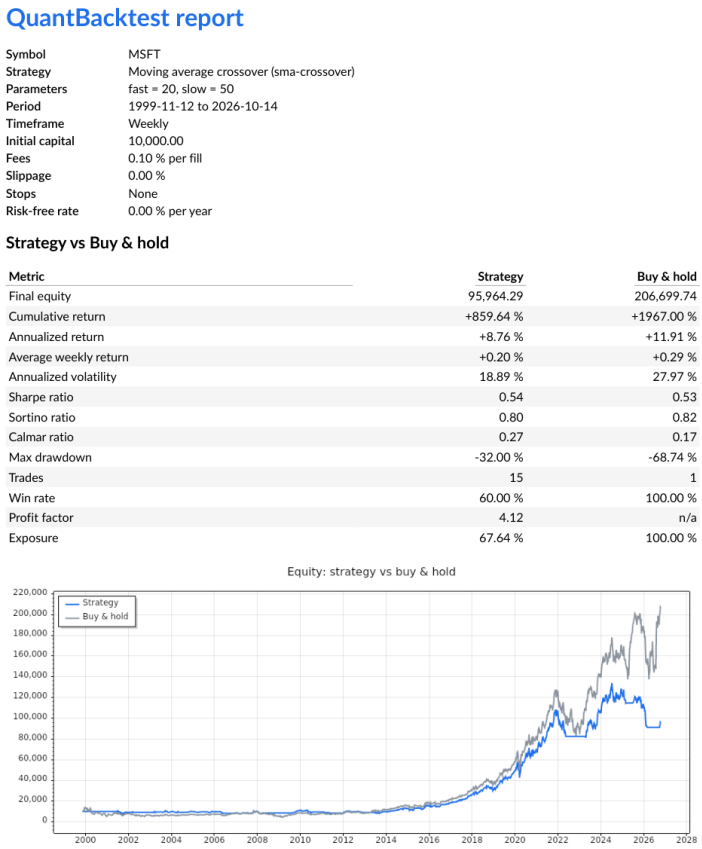
Figure 7.4: The results page at a width of 360 pixels.

Chapter 8

Reports and deployment

8.1 PDF report

The report is generated with QuestPDF (Community license), and its charts are drawn as images with ScottPlot. Every number is printed as it comes in `BacktestResult` and `MonteCarloResult`; the report recomputes nothing. It therefore cannot disagree with the screen. Dates and numbers are always formatted in en-US, whatever the configuration of the computer, and the date in the footer comes from the injected clock, which makes tests reproducible.



QuantBacktest report

Symbol	MSFT
Strategy	Moving average crossover (sma-crossover)
Parameters	fast = 20, slow = 50
Period	1999-11-12 to 2026-10-14
Timeframe	Weekly
Initial capital	10,000.00
Fees	0.10 % per fill
Slippage	0.00 %
Stops	None
Risk-free rate	0.00 % per year

Strategy vs Buy & hold

Metric	Strategy	Buy & hold
Final equity	95,964.29	206,699.74
Cumulative return	+859.64 %	+1967.00 %
Annualized return	+8.76 %	+11.91 %
Average weekly return	+0.20 %	+0.29 %
Annualized volatility	18.89 %	27.97 %
Sharpe ratio	0.54	0.53
Sortino ratio	0.80	0.82
Calmar ratio	0.27	0.17
Max drawdown	-32.00 %	-68.74 %
Trades	15	1
Win rate	60.00 %	100.00 %
Profit factor	4.12	n/a
Exposure	67.64 %	100.00 %

Figure 8.1: PDF report, page 1: settings, 13 metrics against buy & hold, and the equity curve (MSFT weekly, SMA 20/50).

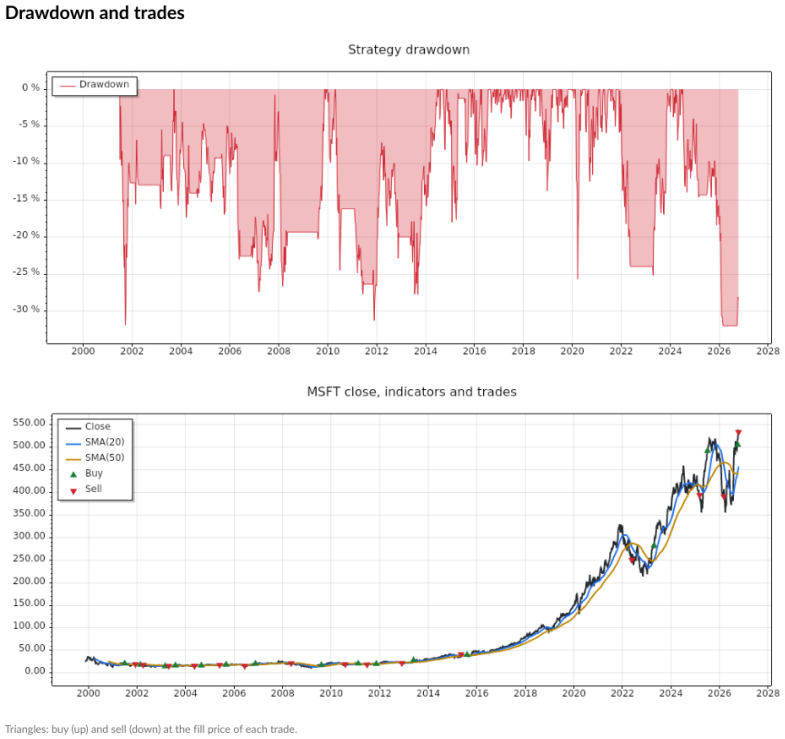


Figure 8.2: PDF report, page 2: drawdown, closes with indicators, and buy and sell markers.

Trade log

#	Entry date	Entry price	Exit date	Exit price	Quantity	Fees	Net profit	Return	Exit reason	Bars held
1	2001-06-29	21.02	2001-12-07	19.41	475.3578	19.22	-781.13	-7.81 %	Signal	23
2	2002-02-22	18.22	2002-04-05	18.20	505.4495	18.41	-30.67	-0.33 %	Signal	6
3	2003-02-28	14.91	2003-04-25	15.63	615.4351	18.80	420.57	+4.58 %	Signal	8
4	2003-08-01	16.44	2004-05-14	15.73	583.8807	18.79	-431.70	-4.49 %	Signal	41
5	2004-08-27	16.78	2005-05-27	17.74	546.2817	18.86	503.65	+5.49 %	Signal	39
6	2005-09-02	18.53	2006-06-09	15.86	521.9014	17.95	-1,412.95	-14.60 %	Signal	40
7	2006-11-17	20.52	2008-05-09	21.42	402.5555	16.88	345.94	+4.18 %	Signal	77
8	2009-08-07	17.52	2010-07-30	19.38	491.2685	18.13	897.39	+10.42 %	Signal	51
9	2011-02-11	21.07	2011-06-24	18.55	450.8895	17.87	-1,155.27	-12.15 %	Signal	19
10	2011-11-11	20.24	2012-11-30	22.04	412.3625	17.44	725.03	+8.68 %	Signal	55
11	2013-05-24	28.22	2015-05-08	41.26	321.4806	22.34	4,170.42	+45.93 %	Signal	102
12	2015-08-07	40.34	2022-05-20	251.05	328.1928	95.63	69,058.59	+521.15 %	Signal	354
13	2023-04-21	282.09	2025-03-07	394.18	291.4923	197.13	32,474.28	+39.45 %	Signal	98
14	2025-07-03	492.15	2026-03-06	391.26	232.9965	205.83	-23,712.21	-20.66 %	Signal	35
15	2026-10-02	505.47	2026-10-14	533.69	179.9928	187.04	4,892.35	+5.37 %	End of data	3

Figure 8.3: PDF report: trade log. With many trades the table continues over several pages and its header is repeated.

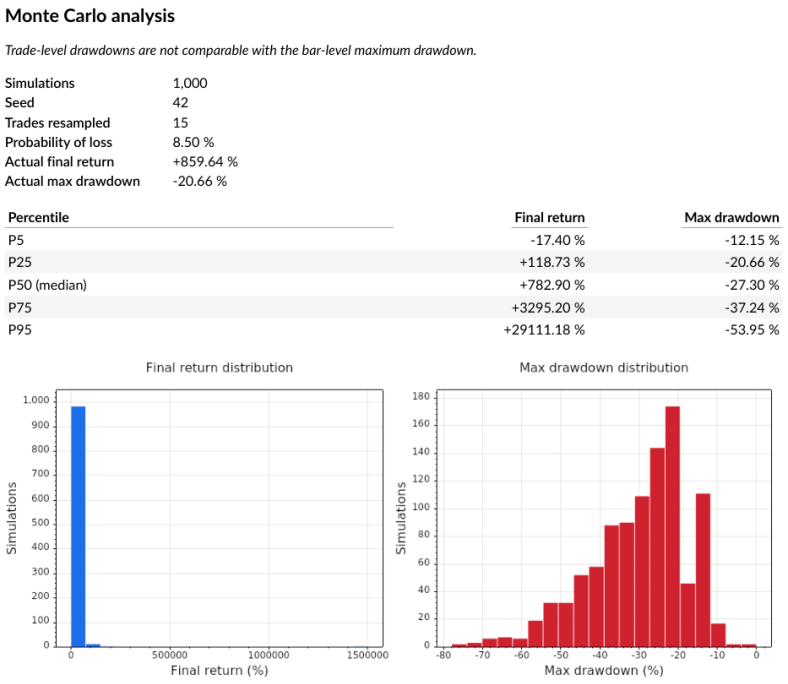


Figure 8.4: PDF report: optional Monte Carlo page.

We checked one real report by hand. For MSFT weekly with the SMA crossover, the 15 net profits add up to 85,964.29, so the final equity is $10{,}000 + 85{,}964.29 = 95{,}964.29$, which is the printed value. The cumulative return, the annualized return, the Calmar ratio, the win rate and the profit factor also match when recomputed from the printed values.

The trades can also be downloaded as CSV. The format is fixed: invariant culture, `\n` line endings, and a header that is always present.

8.2 Docker

The application starts with a single command:

```
cp .env.example .env          # optional: put the Alpha Vantage key in it
docker compose up --build     # then open http://localhost:8080
```

The `Dockerfile` has two stages. The .NET 10 SDK stage restores the packages, builds with warnings treated as errors and runs every test, so an image can only exist for a solution whose tests pass. The ASP.NET runtime stage runs the application as the non-root `app` user, with the fonts needed by the charts. The SQLite database is in a named volume, the seed files are copied into the image, and the API key comes from a local `.env` file which is never committed.

We tested this with Docker Desktop: the image built with its tests, the application answered on port 8080 as the `app` user, the database was created in the volume, a PDF with its Monte Carlo page was generated, and a real API refresh succeeded.

Chapter 9

Quality and teamwork

9.1 Tests

The solution has 1,494 automated tests, written with xUnit and FluentAssertions. Table 9.1 gives the distribution. They use no network and no real clock, and any randomness has a fixed seed. Each test quotes the identifier of the rule it checks.

Project	Tests	Source lines	Test lines
Web	923	7,127	9,384
Engine	184	1,023	2,427
Strategies	168	2,029	1,662
Data	124	1,201	1,563
Reports	55	771	577
Analytics	14	212	217
Fakes	14	981	144
Core	12	713	70
Total	**1,494**	**14,057**	**16,044**

Table 9.1: Automated tests and lines of C# per project.

Some things are not covered by these tests. Razor views, JavaScript and CSS were checked by hand in a browser, and the text of a PDF cannot be read back by the tests, so it is tested before layout.

9.2 Git workflow and continuous integration

`main` always had to compile and pass its tests. All changes arrived by pull request, from a branch named after its part (`part1/`, `part2/…`), and were merged with “Squash and merge”. Between 13 October and 8 November 2026, 42 pull requests were merged (Alexis 16, Alexandre 14, Victor 6, Eloi 6).

GitHub Actions runs two checks on each pull request (Figure 9.1):

- `build-and-test` compiles in Release with warnings as errors and runs all the tests;

- `pr-rules` checks the format of the title (for example `feat(engine): ...`) and verifies that the changed files belong to the part of the branch. A member working on the engine cannot change the web project by accident.

The branch protection of `main` requires both checks, an up-to-date branch and a linear history, and it forbids force-pushes.

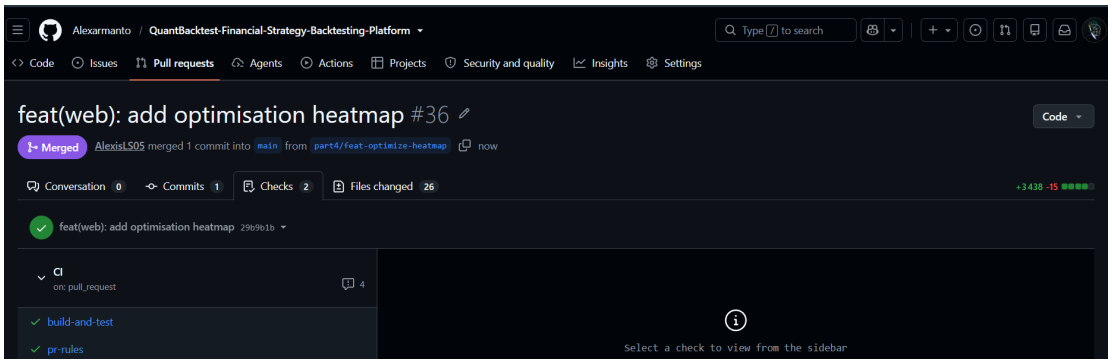


Figure 9.1: Checks of a pull request: `build-and-test` and `pr-rules` are green.

At the start, each pull request needed one approval from another member (Figure 9.2). On 15 October we decided to remove this requirement, because waiting for a review was slowing the team down too much, and we kept the three approvals for any change of the contract. With hindsight, this was a compromise: the CI guarantees that the code compiles and passes its tests, but it does not replace a human reading the code.

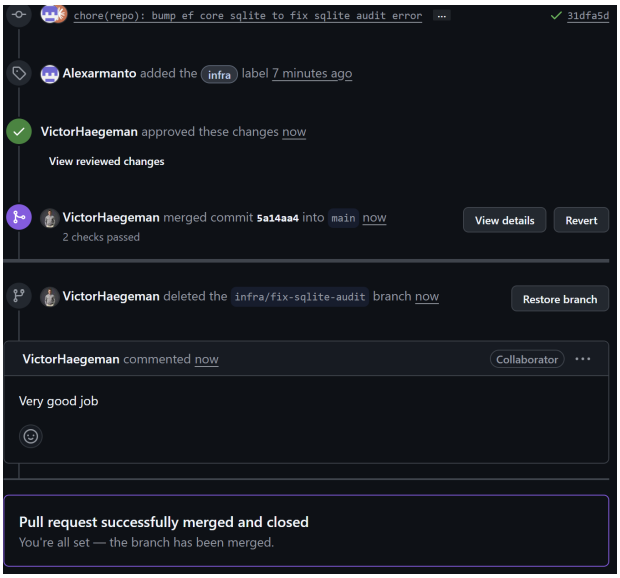


Figure 9.2: An early pull request, approved by another member and merged with its two checks passing.

9.3 Use of AI assistants

We used an AI assistant, Claude Code, throughout the project, and we want to be clear about it. A large part of the code, of the tests and of the text of this report, including this chapter, was drafted with its help. Our role was to define what had to be built, to decide, to check and to take responsibility for the result.

Using an assistant without control quickly produces code that looks correct and is not, so we organised its use around rules that apply to all four of us:

- a `CLAUDE.md` file, the same for everyone, states the conventions and forbids modifying files outside the part of the branch;
- the contract and one specification per part are the reference: a question that the specification does not answer must be asked to a human;
- each test quotes the rule it verifies, so that a reader can compare the test with the specification;
- expected values are computed independently of the code under test, by hand or in a spreadsheet;
- the CI checks everything automatically, for all contributors, human or not.

The checks done by humans remained essential. Real API calls, the run in Docker and the reading of the pages and PDFs on real data found things that tests had not shown, such as a chart font that differed only inside the Docker image.

9.4 What we learned

The most useful decision was the one we took first: spending time on a precise contract before writing code. It allowed four people to work in parallel, and no contract change was ever needed. We also learned that a backtest is easy to get wrong in silent ways, such as a look-ahead, a misread decimal or an error message saved as data. For that reason, the tools that show the limits of a result (train/test, Monte Carlo, the overfitting warning, the data quality report) ended up being as important to us as the strategies themselves.

Chapter 10

Limitations and future work

10.1 Data, reports and deployment

The free plan gives only 100 daily bars, and these are not adjusted, so a split in this window would appear as a price jump. Long tests have to be run in weekly. The seed files are snapshots of October 2026; their last weekly bar is a week that was still in progress, and they are Alpha Vantage data included for this academic project only. If a symbol is refreshed before its seed file has been imported, the seed is bypassed for good. The API quota counter only knows the requests of this application. A future version could use a paid or additional data source to obtain long daily and adjusted histories.

10.2 Strategies and Monte Carlo

Strategies are long only and always give one signal per bar. *Turn of the month* ignores market holidays, which are not known to the application. The no-code builder covers common indicators, but cannot express every rule. The Monte Carlo analysis resamples the trades of a single backtest, and assumes that trades are independent, which ignores market regimes. Its equal-width histograms can be dominated by one extreme value, as in Figure 8.4.

10.3 Engine

There is no short selling and no leverage. Outside protective exits, orders are filled at the next Open without any volume limit, which is optimistic for large orders. With daily or weekly bars, the order of the High and the Low inside a bar is unknown; the “stop first” assumption is prudent, but it remains an assumption. Cash earns no interest, and taxes are ignored. Modelling these would be the first step towards a more realistic simulator.

10.4 Web application

Only the equity chart has a logarithmic scale, so old prices are squashed near zero on long histories. The comparator uses default parameters only. The best cell of a heatmap may rest on very few trades. Train/test uses a single split: walk-forward validation, with several successive splits, would be a natural improvement. The CSV uses a comma and a decimal point, as the contract requires,

so a French version of Excel puts every value in one column. Views and JavaScript have no automated tests.

10.5 Method

Our demonstration symbols are large, successful US companies and an index, chosen with hindsight. This is a survivorship and selection bias. On such symbols buy & hold is very hard to beat, which says more about the symbols than about the strategies. And in all cases, past results do not predict future results.

Appendix A

User manual

A.1 Getting started

With Docker (recommended)

1. Install Docker Desktop and start it.
2. In the project folder, copy `.env.example` to `.env`. Optionally, write your free Alpha Vantage key after `ALPHAVANTAGE_API_KEY=`.
3. Run `docker compose up -build`. The first build takes several minutes, because it also runs the tests.
4. Open `http://localhost:8080`.

Without Docker

With the .NET 10 SDK installed:

```
dotnet user-secrets set "AlphaVantage:ApiKey" "<your key>" \n    --project src/
    QuantBacktest.Web
dotnet run --project src/QuantBacktest.Web     # http://localhost:5080
```

Without a key, the application still works on AAPL, MSFT, SPY and JPM thanks to the seed files. Only downloads from Alpha Vantage are unavailable.

A.2 Fetching and refreshing data

A series is loaded the first time it is used. To get newer bars, open **Data**, enter a symbol and a timeframe under “Refresh a series from Alpha Vantage”, and confirm. Each refresh uses one of the 25 daily requests, and the badge at the top right shows how many were used today.

A.3 Running a backtest

On **Backtest**, choose a symbol, a timeframe and a strategy, and adjust its parameters. Dates are optional: by default the whole series is used. Then set the capital, fee, slippage and risk-free rate. Stop-loss, take-profit and trailing stop are disabled when left empty. Click **Run backtest**.

A.4 Reading the results

The table compares the strategy with buy & hold. Look at several metrics together: a high return with a very large drawdown may be unbearable in practice, and a strategy with few trades proves little. “n/a” means that a ratio is mathematically undefined. The trade log gives the reason of each exit: Signal, Stop-loss, Take-profit, Trailing stop or End of data. If a banner says that prices are not adjusted, splits and dividends are not taken into account.

A.5 Strategies

Strategies lists all strategies. Each card gives the idea, the exact rules, the risks and the parameters, and a button starts a backtest with it.

A.6 Building a strategy without code

On **Strategies**, create a custom strategy and give it a name. Add 1 to 5 conditions for the entry and for the exit, and choose whether all or any of them must be true. If entry and exit are true on the same bar, the signal is a sale. The strategy then appears in every list. Be careful: saving changes to a strategy also changes what its existing links compute.

A.7 Writing a strategy in C#

Copy `src/QuantBacktest.Strategies/Custom/TemplateStrategy.cs` into the `Strategies` folder, rename the class, remove the `[HiddenStrategy]` line, choose a unique identifier in lower case with dashes, and fill in the documentation, the parameters and the rules. Rebuild the application and the strategy appears automatically. A strategy must only look at bars up to the current one, and must return `Hold` while its indicators are not ready.

A.8 Comparing strategies

Compare runs 2 to 5 strategies with their default parameters on the same data and shows their curves together.

A.9 Optimising parameters

Optimize tests all pairs of values of two parameters (the strategy needs at least two) and colours the results. Click on a cell to open its backtest. An “isolated peak” banner means the best result is much better than its neighbours, which is a classic sign of overfitting. Prefer a stable area to a lonely peak.

A.10 Train/test validation

Validate optimises on the first part of the period and tests the best parameters on the rest, which the optimisation has not seen. Compare the two columns: a strong fall in the test period is a warning.

A.11 Monte Carlo

From a result, click **Run Monte Carlo**. Choose the number of simulations (100 to 10,000) and a seed. The same seed always gives the same result. At least 5 trades are required. Its drawdown is measured trade by trade and cannot be compared with the maximum drawdown of the result page.

A.12 Data quality and API quota

On **Data**, each series has a quality report listing jumps, zero volumes, stale prices and gaps. A flag does not necessarily mean an error: many are real market events.

A.13 Exporting reports

On a result page, **Download PDF** gives the full report and **Download trades CSV** the trade log. On the Monte Carlo page, **Download PDF with Monte Carlo** adds the Monte Carlo section.

A.14 Troubleshooting

Message or problem	What to do
API key not configured	Set the key in user-secrets, or in `.env` with Docker, then restart. Or use a symbol that has a seed file.
Request limit reached	The free plan allows 25 requests per day, reset at midnight UTC. Use cached symbols in the meantime.
Symbol not found	Check the symbol on Alpha Vantage.
Prices are not adjusted	Normal for daily data on the free plan. Use weekly for long tests.
Not enough trades for Monte Carlo	At least 5 trades are needed. Use a longer period or another strategy.
`docker` is not recognised	Docker Desktop is not installed, or the terminal was opened before its installation.